Object-Oriented Database OODB & Cloud Computing

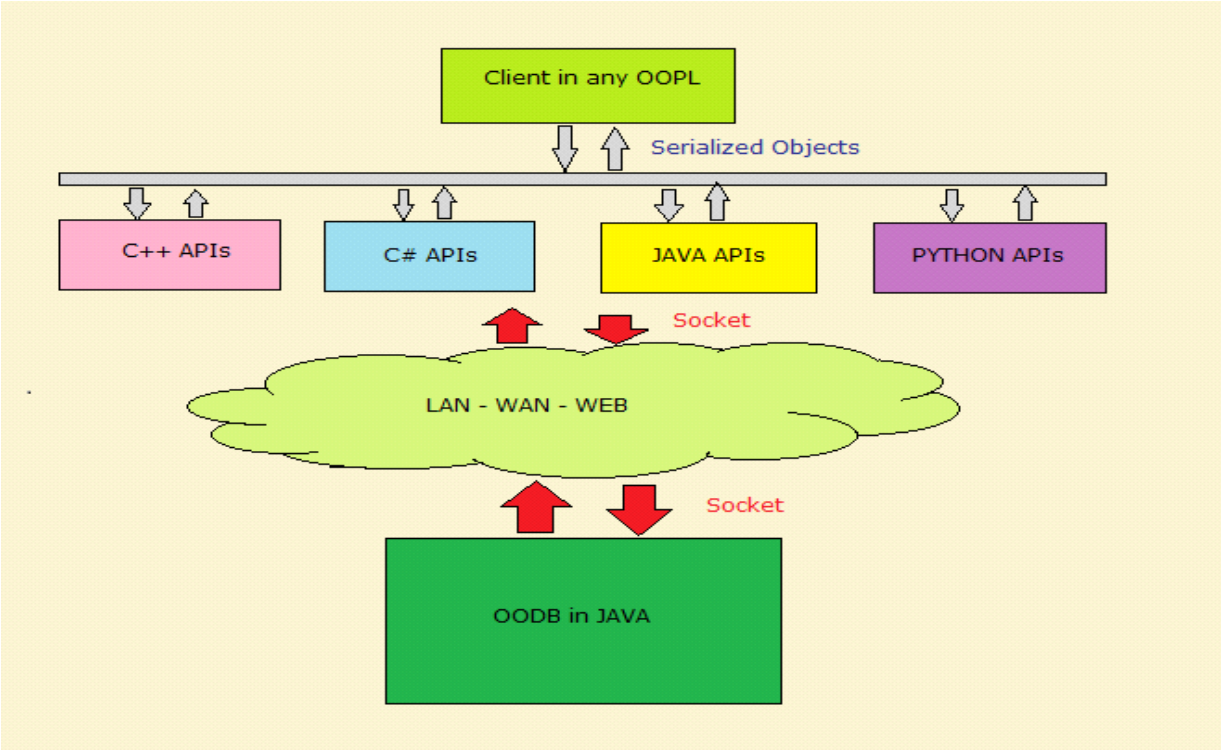
FOREWORD

Object-oriented programming languages (OOPL) are the preferred programming languages in software application development today. C++, C#, PYTHON and JAVA are usually the first OOPL choices. Accordingly, the OOPL application data are objects that can be simple (text) or complex (e.g. with images, sounds). However, the main problem with OOPL objects is how to store those objects and how to retrieve them exactly as they originally were. With traditional relational databases (RDB) such as MySQL or Oracle, the objects must be atomized in such a way that they can only be retrieved in SQL (Structured Query Language). And that's the problem. Atomized objects are unsuitable for object implementation. Since OOPL objects are OOPL dependent, they are usually incompatible with each other, making a C# object useless in JAVA or vice versa. Therefore, it is almost impossible to design and build an object-oriented database that should be generic for every kind of OOPL objects.

OBJECT-ORIENTED DATABASE (OODB)

Attempting to create a generic object-oriented database (OODB) in an object-oriented programming language (OOP) requires a physical separation of storage and access mechanisms. The most common classic model is the client-server model. Here, the server can be implemented in any OOP language (in this case, Java), and the object data is stored unchanged as a byte array. This allows the development of clients in any OOP language, each with its own storage mechanism (serialization) and data model for the object. The storage data format is the object's serialization, which can be stored as a byte array on any storage medium, allowing for later retrieval at any time. This forms the basis for implementing an OODB.

OODB ARCHITECTURE



As mentioned earlier, serialized objects are object-oriented and distinct from one another. The data must be bundled into a byte array before it can be stored or retrieved (i.e., written to or read from a file).

Instead of writing to or reading from a file, an implemented object-oriented client API sends or receives the serialized data byte by byte to or from a socket connected to an OODB server via a network (e.g., the internet). This communication method physically separates clients from the server and allows clients to be implemented in any object-oriented programming language using the appropriate API: The client serializes an object, stores the serialized data as an object frame using the Serializable API, and retrieves it using the same object frame as if reading/writing a virtual file, while the client APIs handle the sending and receiving communication between clients and the OODB server.

Some examples in C#, Java, and Python.

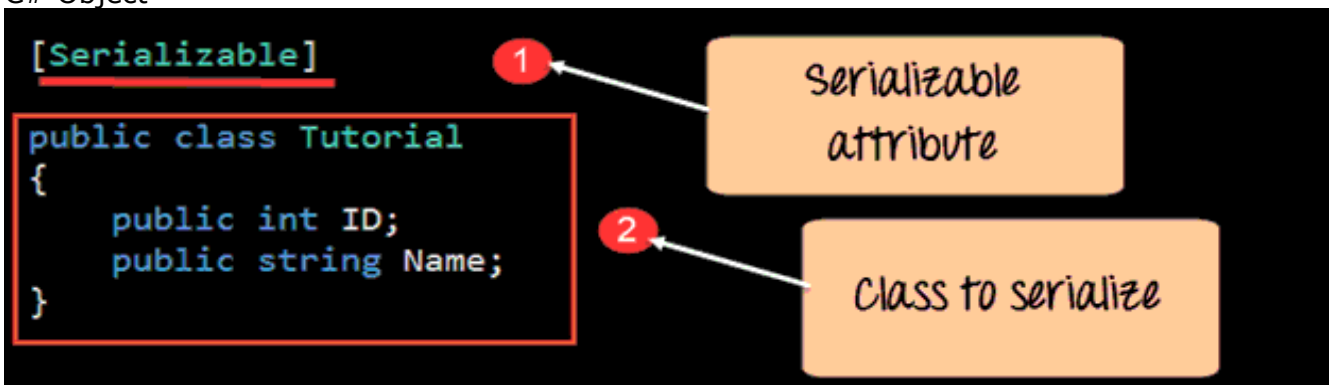
C# (C SHARP) (Source: https://www.guru99.com)
The codes

```
using System;
using System.IO;
using System.Linq;
using System.Runtime.Serialization;
using System.Runtime.Serialization.Formatters.Binary;
using System.Text;
using System.Threading.Tasks;
namespace C_SharpExample
{
  [Serializable]
  class Tutorial
  {
  public int ID;
  public String Name;
   static void Main(string[] args)
   {
    // create and initialize
    Tutorial obj = new Tutorial();
```

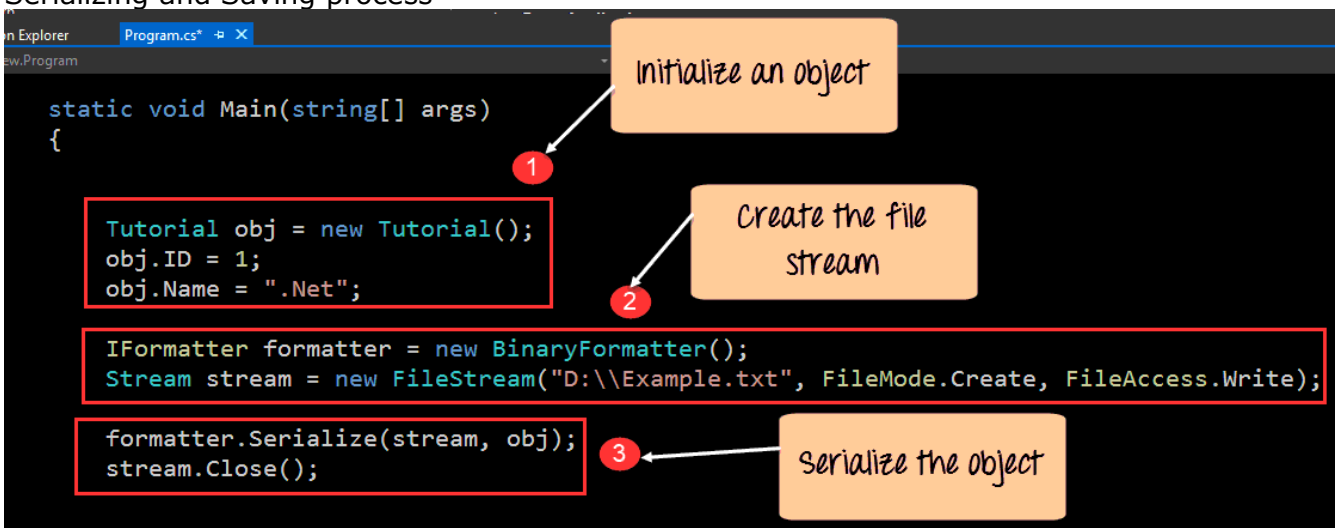
```
    obj.ID = 1;
    obj.Name = ".Net";
    // save or write to D:\Example.txt
    IFormatter formatter = new BinaryFormatter();
    Stream stream = new FileStream(@"D:\Example.txt",FileMode.Create,FileAccess.Write);

    formatter.Serialize(stream, obj);
    stream.Close();
    // retrieve or read from D:\Example.txt
    stream = new FileStream(@"D:\Example.txt",FileMode.Open,FileAccess.Read);
    Tutorial objnew = (Tutorial)formatter.Deserialize(stream);

    Console.WriteLine(objnew.ID);
    Console.WriteLine(objnew.Name);

    Console.ReadKey();
  }
 }
}
```

Explanation:

C# Object



Serializing and Saving process



Reading (or deserializing) process

```
stream = new FileStream("D:\\Example.txt", FileMode.Open, FileAccess.Read);

Tutorial objnew = (Tutorial)formatter.Deserialize(stream);

Console.WriteLine(objnew.ID);
Console.WriteLine(objnew.Name);

Console.ReadKey();
}
```

1 Create the file stream

2 Deserialize the object

3 Write the data to the console

JAVA

The codes

```
// the object
public class Tutorial  implements java.io.Serializable {
  public int id;
  public String name;
}
// the application
import java.io.*;

public class JavaExample {
  public static void main(String[] args) throws Exception {
      // create and initialize
      Tutorial obj = new Tutorial();
      obj.id = 1;
      obj.name = "java";
      // save or write to C:\example.txt
      ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("Example.txt", false));
      oos.writeObject(obj);
      oos.close();
      // retrieve or read from C:\example.txt
      ObjectInputStream ois = new ObjectInputStream(new FileInputStream("Example.txt"));
      Tutorial tut = (Tutorial) ois.readObject();
      ois.close();

      System.out.println("ID:"+tut.id+"Name:"+tut.name);
  }
}
```

Explanation

Java Object

```
public class Tutorial  implements java.io.Serializable {
  public int id;
  public String name;
}
```

```
public class JavaExample {
  public static void main(String[] args) throws Exception {
    // create and initialize
    Tutorial obj = new Tutorial();
    obj.id = 1;
    obj.name = "java";
    // save or write to C:\example.txt
    ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("Example.txt",
false));
    oos.writeObject(obj);
    oos.close();
    // retrieve or read from C:\example.txt
    ObjectInputStream ois = new ObjectInputStream(new FileInputStream("Example.txt"));
    Tutorial tut = (Tutorial) ois.readObject();
    ois.close();

    System.out.println("ID:"+tut.id+" Name:"+tut.name);
  }
}
```

Create and initialize

Save object

Read Object

PYTHON

The codes (PYTHON 3)

```
import pickle
tutorial={"id":":1, "name:":"Python"}

f=open("examplePython.txt","wb")
pickle.dump(tutorial,f)
f.close()

f=open("examplePython.txt","rb")
d=pickle.load(f)
print (d)
f.close()
```

Explanation

```
import pickle
tutorial={"id":":1, "name:":"Python"}
```

Serialize API and Object initialization

```
f=open("examplePython.txt","wb")
pickle.dump(tutorial,f)
f.close()
```

Save object using pickle

```
f=open("examplePython.txt","rb")
d=pickle.load(f)
print (d)
f.close()
```

Read object using pickle

The serialization algorithm shows us how an OOPL object is created, initialized, and stored, and how it differs from OOPL to OOPL. The following content of the PYTHON and JAVA serialized object highlights the difference between PYTHON and JAVA "serialized" data:

Python 3

```
0        4        8        A        E
80037071 00285803 00000069 643A7101 4B015805 | ..}q .(X. ...i d:q. K.X. | 0
0000006E 616D653A 71025806 00000050 7974686F | ...n ame: q.X. ...P ytho | 1
6E710375 2E                                  | nq.u .                  | 2
```

JAVA

```
0        4        8        A        E
ACED0005 73720008 5475746F 7269616C 2C87FB20 | .... sr.. Tuto rial ,..  | 0
47BBD73C 02000249 00026964 4C000046 61606574 | G..< ...I ..id L..n amet | 1
00124C6A 6176612F 6C616E67 2F537472 696E673B | ..Lj ava/ lang /Str ing; | 2
78700000 00017400 046A6176 61                | xp.. ..t. .jav a         | 3
```

As we can see, serialized OOPL data varies considerably, making conversion between different OOPL objects difficult and time-consuming. To deserialize serialized objects with executable methods, OOPL always requires an object frame (i.e., a class) that transforms serialized object data into an executable object instance. Therefore, all serialized object frames must always be bundled with the client application.

Examples: **C#** object with some internal methods

C# TestObject

```
public class TestObject {
   private int age;
   private string name;
   static string companyname;
   public int Age {
      get {
         return age;
      }
      set {
         age = value;
      }
   }
   public string Name {
      get {
         return name;
      }
      set {
         name = value;
      }
   }
   public static string CompanyName {
      get {
         return companyname;
      }
      set {
         companyname = value;
      }
   }
}
```

```
using System;
using System.IO;
using System.Runtime.Serialization;
using System.Runtime.Serialization.Formatters.Binary;
public class SerialTest {
   public void OnSerialize() {
      TestObject c = new TestObject();
      c.Name = "bipin";
      c.Age = 26;
      TestObject.CompanyName = "xyz";
      File f = new File("temp.dat");
      Stream s = f.Open(FileMode.Create);
      BinaryFormatter b = new BinaryFormatter();
      b.Serialize(s, c);
      s.Close();
   }
   public void OnDeserialize() {
      TestObject c = new TestObject();
      File f = new File("temp.dat");
      Stream s = f.Open(FileMode.Open);
      BinaryFormatter b = new BinaryFormatter();

      c = (TestObject) b.Deserialize(s);

      Console.WriteLine("Name :" + c.Name);
      Console.WriteLine("Age :" + c.Age);
      Console.WriteLine("Company Name :" + TestObject.CompanyName);
    }


   public static void Main(string[] s) {
      SerialTest st = new SerialTest();
      st.OnSerialize();
      st.OnDeserialize();
   }
}
```

Serialized TestObject

Deserialize and Cast to TestObject

JAVA Object with some methods

```
public class TestObject {
   private int age;
   private string name;
   static string companyname;
   public int getAge() {
      return age;
   }
   public void setAge(int value) {
      age = value;
   }
   public string getName() {
      return name;
   }
   public void setName(String value) {
      name = value;
   }
   public string getCompanyName {
      return companyname;
   }
   public void setCompanyName(String value) {
     companyname = value;
   }
}
```

Java TestObject

```
#import java.io;

public class SerialTest {
   public void OnSerialize() throws Exception {
      TestObject c = new TestObject();
      c.setName = "bipin";
      c.setAge = 26;
      c.setCompanyName = "xyz";
      ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("temp.dat", false));
      oos.writeObject(c);
      oos.close();
   }

   public void onDeserialize() throws Exception {
      TestObject c = new TestObject();
      ObjectInputStream ois = new ObjectInputStream(new FileInputStream("temp.dat"));

      c = (TestObject) ois.readObject();

      System.out.println("Name :" + c.getName);
      System.out.println("Age :" + c.getAge);
      System.out.println("Company Name :" + c.getCompanyName);
      ois.close();
   }
   public static void main(String[] s) {
      SerialTest st = new SerialTest();
      st.OnSerialize();
      st.OnDeserialize();
   }
}
```

Serialize TestObejct

Deserialize and Cast TestObject

OODB CLIENT-SERVER ARCHITECTURE

From the client application's perspective, object data is dependent on the object-oriented programming language (OOPL) used. Converting or deserializing serialized data between different OOPLs is not practical, as the process is not only difficult but also highly error-prone. It is better to leave the serialized data unchanged and store it independently in an object-oriented database (OODB). The client application can then use its object frame to reassemble (deserialize) the serialized data from the OODB back into the original, OOPL-dependent object instance. This allows an OODB, which can be implemented in any OOPL, to store and retrieve the data serialized (in binary format) by any OOPL used by clients.

On the server side, the implementation can be done in any object-oriented programming language (OOPL). The physical separation between clients and server ensures an OOPL-independent implementation. However, the OOPL-dependent data must be known to and accessible by the clients exclusively within that OOPL, since the data represents only an array of binary bytes to the server.

The server network allows clients to directly access OODB on any server or simultaneously on multiple servers within the network. An implemented OODB event enables each server in the network to determine the status of other servers – that is, which servers are running and which are down. Furthermore, each server must be able to access other servers on behalf of the client if the OODB system is distributed network-wide (i.e., across different servers within the network).

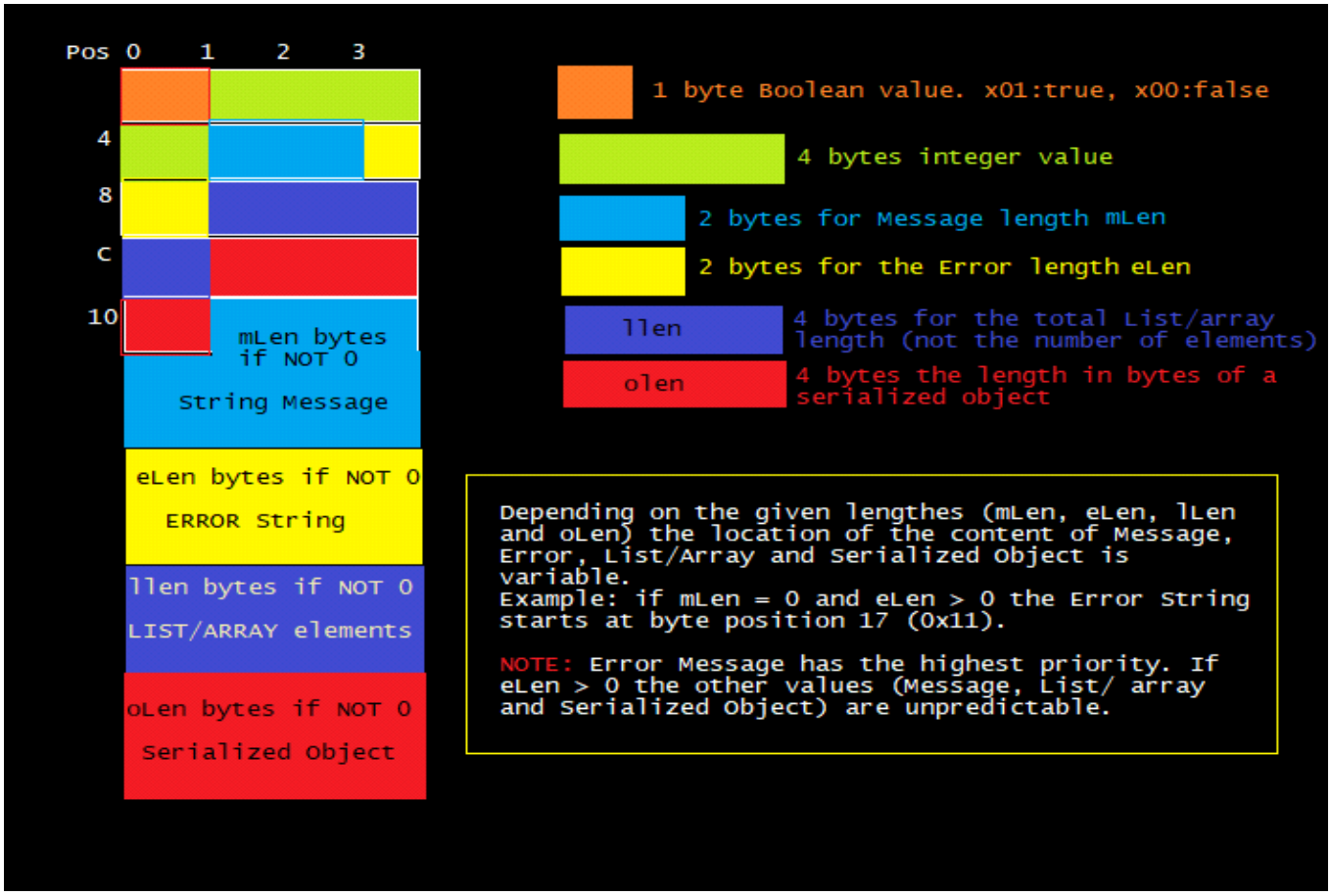
CAUTION

To access a local OODB server from outside the network, the server's router (using **IPv4**) must be configured to forward the server port, or **IPv6** can be used. Any client outside the network can access the website via its web name or IP address.

A client API package written in an object-oriented programming language other than Java must implement the reception of the byte stream (as a byte array) in the corresponding object-oriented programming language (e.g., Python, C#, etc.) from the following byte stream layout:

```
The Receiving Data Stream Buffer Layout
-------------------------------------------------------------------------------------
 Byte Pos.   length-in-bytes   name    OBBStream
   0:              1           bool    contains bool x01 for true, x00 for false
   1-4:            4           Int     contains an Int value or 0 if no Int is needed
   5-6:            2           msg     contains msg length, mLen can be 0
   7-8:            2           err     contains err length, eLen can be 0
   9-12:           4           list    contains list length (not the size), lLen can be 0
   13-16:          4           obj     contains the byte array length. bLen can be 0
---------------- 17 bytes -----------------------------------------------------------
   17              mLen                start of msg String
   a = 17+mLen     eLen                start of err String
   b = a+eLen      lLen                start of list or array of all String items (see note)
   c = b+lLen      bLen                start of byte array
 len = c+bLen                          EOF (the content of Stream)
--- Total in bytes: len = 17+mLen+eLen+lLen+bLen -------------------------------------

If xLen = 0 item x is NULL and the next Item takes this position. Exp. a = 17 if
mLen is 0 and eLen is not 0 then the err starting pos. is 17. The size is variable.

Note: each list/array element is started by 2 bytes indicating the length of the element
and the element itself in bytes. The element String can be either literal or binary
```

Data persistence in a database is typically stored in a special file that enables fast access and ensures consistency. In Java, such a file is the RandomAccessFile (RAF), which, in addition to fast access, also provides consistency through file locking via FileChannel. However, the RAF is generic and must be adapted for object-oriented architectures (OODB) with access via object names.



For example: A serialized object that was read from the server by the client and is error-free has the layout with the following content (no message, no error, and no further byte array):

- byte 5-6: 2 bytes (short), message length, also both = 0x00 → mlen = 0
- byte 7-8: 2 bytes (short), error length, also both = 0x00 → eLen = 0
- byte 9-12: 4 bytes, (int) byte array length, all = 0x00 → lLen = 0
- byte 13 – 16: 4 bytes (int) specifying the size or length of the object byte array (objLen)
- The starting position of the object byte array is 17+0+0+0, and its length is objLen.

The object (obj) is the serialized object in a byte array that starts at position 17 of size objLen and can be deserialized into an object instance using the associated object frame.

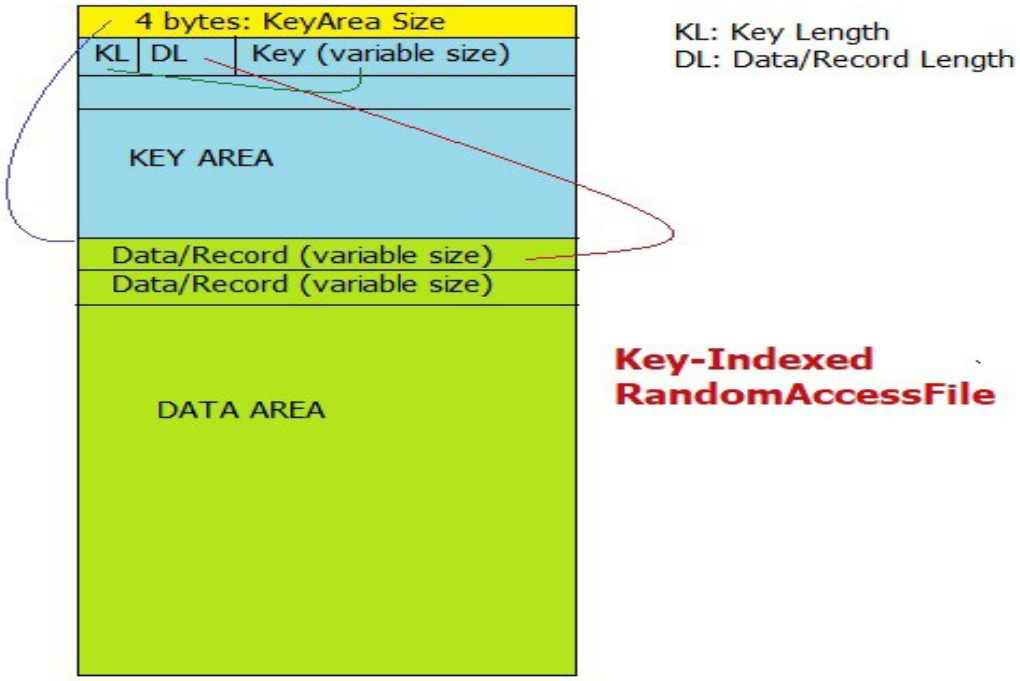
STORAGE (SERVER)

Data persistence in a database is typically stored in a special file that enables fast access and ensures consistency. In Java, such a file is the **RandomAccessFile** (RAF), which, in addition to fast access, also provides consistency through file locking via **FileChannel**. However, the RAF is generic and must be adapted for object-oriented architectures (OODB) with access via object names.

```
RandomAccessFile raf = new RandomAccessFile(dbName, "rw"); // create/open a dbName
FileLock fLocked = raf.getChannel().lock();                // locked it
```

The storage structure consists of **three areas**:

- **Pointer area** has a size of 4 bytes (int) that points to the starting position of ObjectData area.
- **ObjectNames (or Keys) area** contains AccessEntries. Each AccessEntry consists of: 2 bytes (short) for the length of ObjectName, 4 bytes (int) for the size of serialized object (in the ObjectData area) and the ObjectName itself.
- The serialized objects are stored sequentially in the **ObjectData area**. The size of each object is specified in the object name area.



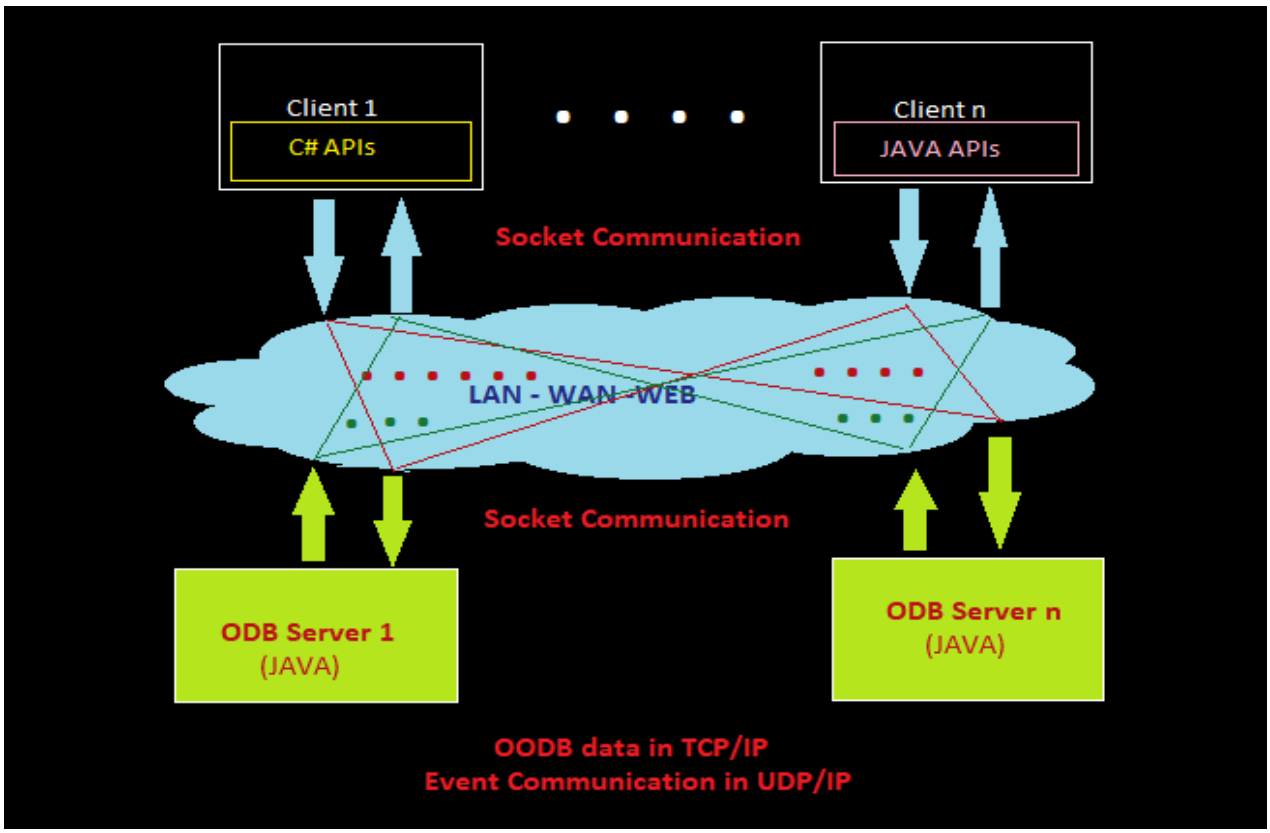
The ObjectNames area is fully cached, ensuring fast access to ObjectName and Object Attributes, while the ObjectData is partially or fully cached depending on the specified **cache buffer** and OODB size (4 + ObjectNames/Keys area + ObjectData area). However, uncached ObjectData is cached when read, updated, or added (by expanding the cache area). Here is the implementation **NanoDB:**

Note: Up here, the short form **Key** is used instead of ObjectName.

```
public class NanoDB {
/**
 constructor
 @param dbName  String, file name
 @param limit  int, max. size to be cached in MB
 @param charsetName String, character set name (e.g. "UTF-8");
 */
 public NanoDB(String dbName, String charsetName, int limit) {
   this.limit = limit;
   this.dbName = dbName;
   cs = Charset.forName(charsetName);
 }
 ...
 /**
 readObject
 @param key String
 @return byte array which can be an array of (non)serialialized object
 @exception Exception thrown by JAVA
 */
 // check for existed key is done by ODBManager
 public synchronized byte[] readObject(String key) throws Exception {
   if (cache.containsKey(key)) return cache.get(key);
   // not in cache. Read from file via size and pointer
   byte[] buf = new byte[sizes.get(key)];
   raf.seek(pointers.get(key));
   raf.read(buf); // fetch
   cache.put(key, buf);
   return buf;
 }
 ...
}
```

And on the client side, the readObject implementation is written in Java, for example.

```
 /**
 read() a seralized object from JODB dbName
 @param dbName String, the DB name.
 @param key String
 @return serialized object or a serialized POJO object if data aren't serialized. See getBytes().
 @exception Exception thrown by JAVA
 */
 public Object read(String dbName, Object key) throws Exception {
   byte[] bb = send(dbName, 10, key);
   if (bb[0] == (byte)0xAC && bb[1] == (byte)0xED) { // serialized object
     ObjectInputStream oi = new ObjectInputStream(new ByteArrayInputStream(bb));
      Object obj = oi.readObject();
     oi.close();
     return obj;
   }
   return bb; // not a serialized POJO
 }
```



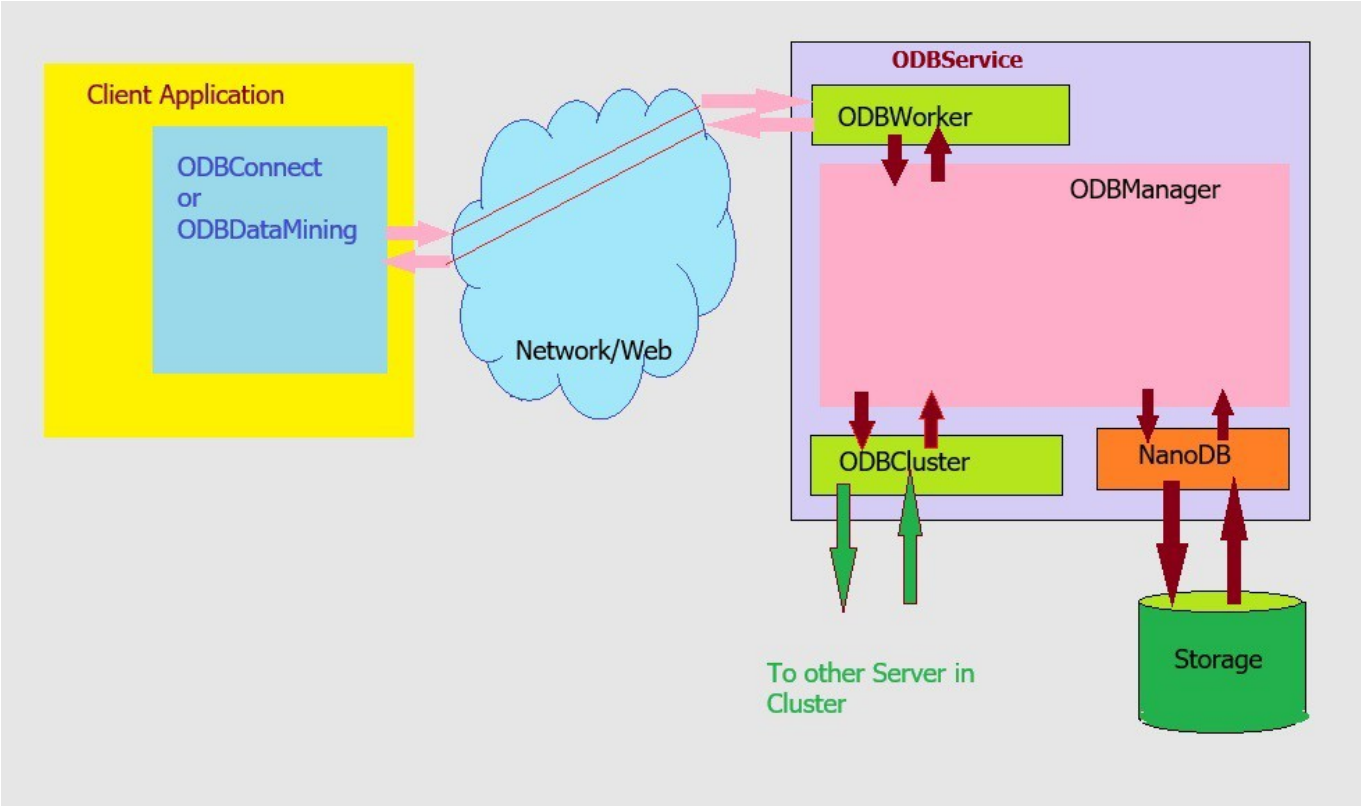
In multi-user environments, it is essential not only to synchronize various access activities but also to coordinate them using lock, unlock, commit, and rollback. The Java keywords "synchronized" and "locked" are insufficient for this purpose. Like relational databases, object-oriented databases (OODBs) must also allow objects to be locked and unlocked for updating or deleting. Furthermore, modified objects must be able to be rolled back to their previous state, or the changes must be permanently saved through a commit. This requires additional management. For example, checking the existence of keys, maintaining a list of updated and deleted objects, a list of uncommitted objects, etc.

OODB IMPLEMENTATION

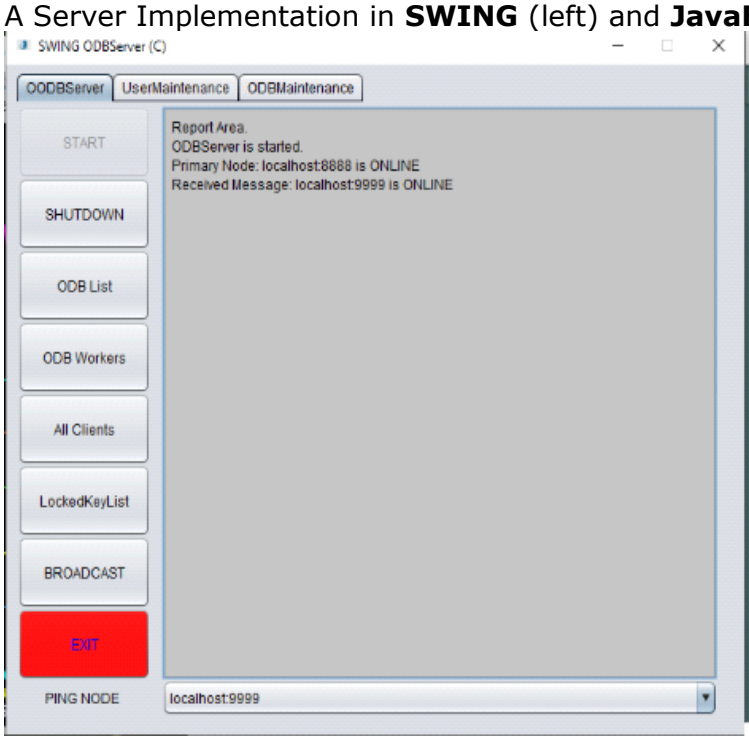
The JAVA OODB package is a bundle of the following JAVA APIs:

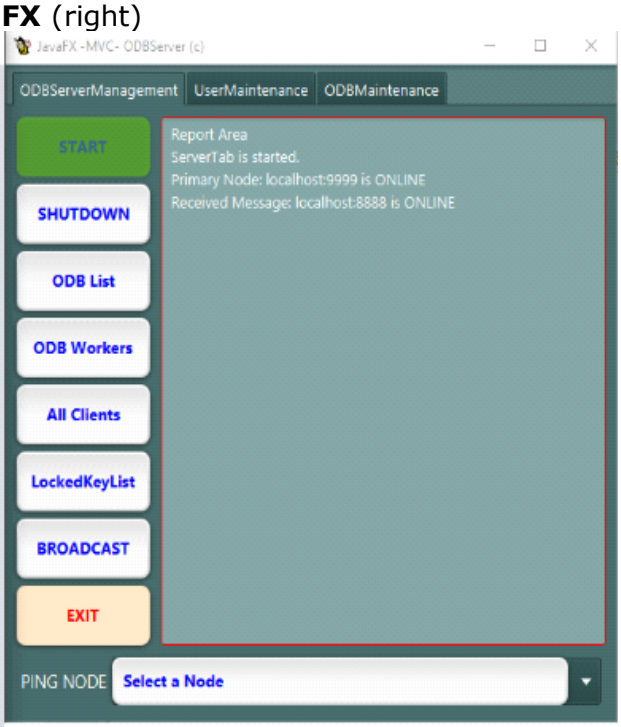
EnDecrypt	Proprietary Encrypting & Decrypting algorithm.
JFXOptions	JavaFX interactive tools
JOptions	Java SWING interactive tools
NanoDB	Data Management System (physical IOs)
ODBBroadcaster	Chuster broadcaster
ODBCluster	Cluster connection
ODBConnect	Client API connection
ODBEvent	Event
ODBEventListener	Cluster Event Listener
ODBEventListening	Interface for Event implementation
ODBInputStream	An extension of InputStream
ODBIOStream	OODB proprietary IO stream
ODBManager	Additional OODB management
ODBDataMining	An extension of ODBConnect

ODBObjectView	Java Serialized Object interpreter
ODBParser	A parser
ODBService	A Server API
ODBWorker	An interface zwischen ODBManager and ODBConnect
UserList	Server's



A Server Implementation in **SWING** (left) and **JavaFX** (right)

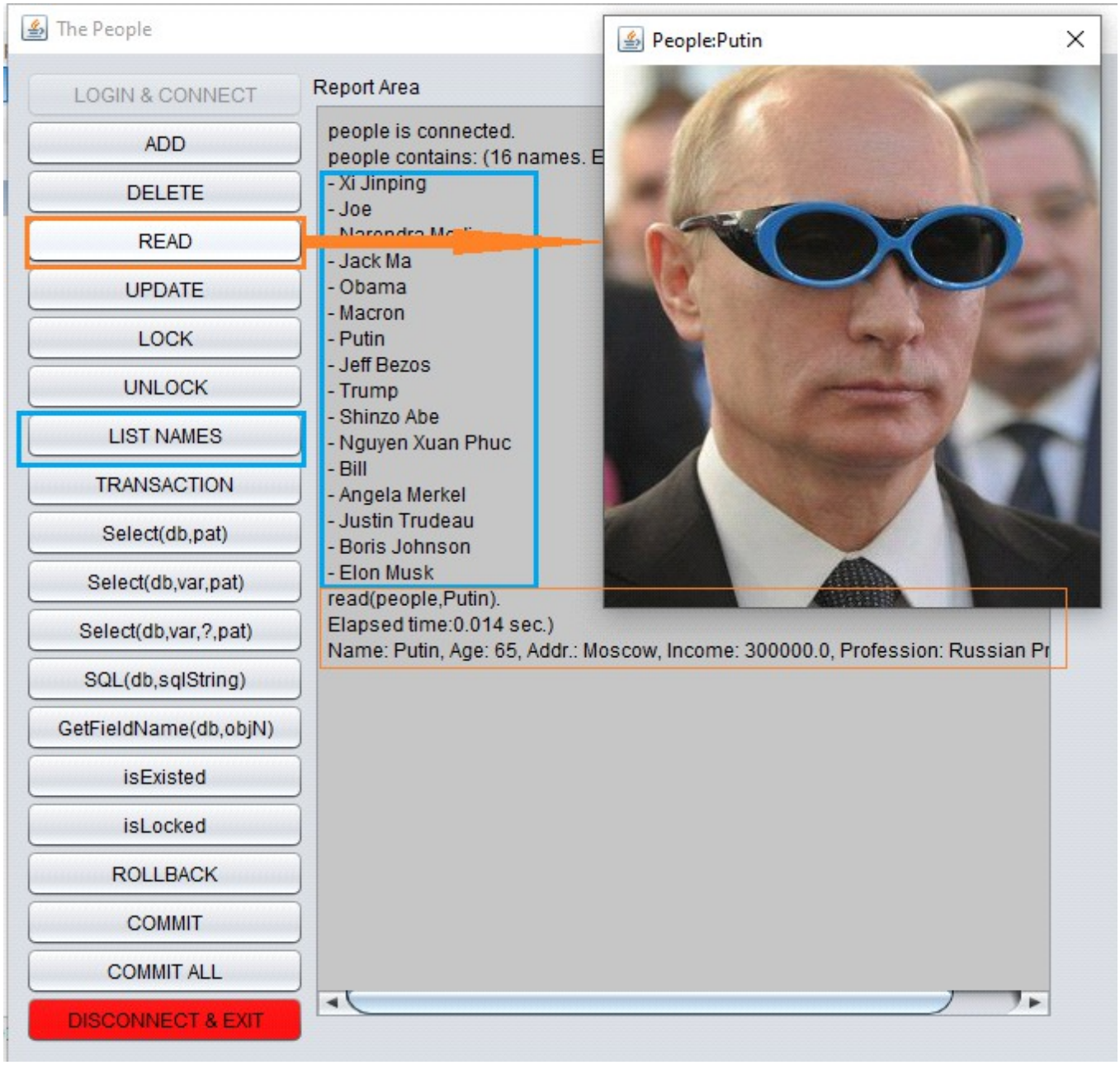




CLIENT APIs

To connect to the ODB server, the client application requires the generic API **ODBConnect**, which provides all the necessary basic methods such as readObject, updateObject, deleteObject, getKeys, etc. Additionally, the Java object-specific API **ODBDataMining** is only needed for certain data mining tools such as select, SQL-like expressions, etc.

An Example with a complex Java Serializable Object. The Object is "People.java" with SWING Dialog Implementation to display Image accessed from the Web:



The People Object

```
public class People implements java.io.Serializable {
  private static final long serialVersionUID = 1234L;
  public People(String name, int age, String address, String profession, double income, String url) {
    this.age = age;
    ...
```

```
  }
  ...
  public void print() {
    ...
  }
  public void picture(JFrame jf) {     // SWING JDialog
    class Picture extends JDialog {
      public Picture(JFrame jf) {
        super(jf, true);
        setTitle("People:"+name);
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        JButton but;
        try {
          ImageIcon img;
          if (url.indexOf("://") >0) img = new ImageIcon(new URL(url));
          else img = new ImageIcon(url);
           if (img != null) but = new JButton(img);
           else but = new JButton("No Image available");
           setSize(img.getIconWidth(), img.getIconHeight());
        } catch (Exception e) {
           but = new JButton("No Image");
           pack();
        }
        but.setContentAreaFilled(false);
        but.setBorderPainted(false);
        add(but, BorderLayout.CENTER);
        setLocationRelativeTo(jf);
      }
    }
    (new Picture(jf)).setVisible(true);
  }
}
```

and the app with **ODBDataMining**:

```
  ...
  // Connect to server: host, port, userID, password
  ODBDataMining dMine = new ODBMining(host, port, login[1], login[0]);
  ...
  ((JButton) map.get("read")).addActionListener(e -> {
    if (!connected) {
      error("There's NO connection to any OODBServer.");
      return;
    }
    String name = JOptions.input(frame, "PeopleName");
    if (name.length() == 0) return;
    if (dMine.isExisted(dbName, name))
      try {
        long beg = System.currentTimeMillis();
        People peo = (People)dMine.read(dbName, name);
        double time = (double)(System.currentTimeMillis()-beg)/1000;
        jta.append("read("+dbName+","+name+").\nElapsed time:"+time+" sec.)\n"+peo.toString()+"\n");
        peo.print( );  // print out
        peo.picture(frame); // display him/her
      } catch (Exception ex) {
        jta.append(getErrorMessage(ex.toString())+"\n");
      }
    else jta.append(name+" is unknown.\n");
    jta.setCaretPosition(jta.getText().length());
  });
```

SERVER API

The OODB package does not include a ready-to-use ODB server, but provides four APIs that must be integrated into your own server.

- **JOptions**: for Swing dialogs/popups
- **JFXOptions:** for JavaFX dialogs/popups
- **ODBService**: The service API starts the ODB server and establishes the interface to the user's server application.
- **UserList**: This API manages registered ODB users.

ODBService requires a **configuration file** in plain text format that provides ODB with the necessary parameters such as cache size, WebHostName/IP address, port, pathes, etc. This configuration file must contain the following elements in **XML**-like syntax:

- **LOGGING**: 0 for No, 1 for Yes
- **MAX_CACHE_LIMIT**: integer between 1MB to 1024MB
- **WEB_HOST/IP**: WebHostName/IP@Port
- **MULTICASTING**: IPv6-Multicast@Port
- **ODB_PATH**: Path to OODB RandomAccessFiles
- **LOG_PATH**: logging path
- **USERLIST**: Path to the UserList (usually shares the same ODB path)

An example:

```
<!this is an example (Case Sensitive)

  - ODB_PATH is the root directory for all ODB
  - LOG is the name of the logging file (abs.path)
  - USERLIST is the encrypted file of all ODB_DMS users
  - WEB_HOST/IP is the WEB name or IP of the Computer Node. Format: name/IP@port
    1) if ?@Port is a question mark ? ODBManager will prompt for IP selection from
       system's IP-List.
    2) if IP4@Port ODBManager will select LAN IPv4 from system's IP-List.
    3) if IP6@Port ODBManager will select Internet IPv6 from system's IP-List.
    4) if name/IP is a string like localhost or similar this name/IP is taken.
  - LOGGING: 0: NO, 1: YES
!>
<LOGGING>0 />
<MAX_CACHE_LIMIT> 2 />
<WEB_HOST/IP>IP6@9999 />
<! Messaging facility for the nodes using the Multicast IP
 The multicast datagram socket class is useful for sending and receiving IP multi-
 cast packets. A MulticastSocket is a (UDP) DatagramSocket, with additional capa-
 bilities for joining "groups" of other multicast hosts on the internet.
 More about MulticastSocket: see MulticastSocket API
 About Global Internet Multicast IP:
    http://www.tcpipguide.com/free/t_IPMulticastAddressing.htm
 for IPv6
    https://www.iana.org/assignments/ipv6-multicast-addresses/ipv6-multicast-addresses.xhtml
!>
<MULTICASTING> FF0E:0:0:0:0:0:0:1@7777 />
<ODB_PATH>C:/joeapp/ODB_Nano/Nodes/oodb_Node1 />
<LOG_PATH>C:/joeapp/ODB_Nano/Nodes/log_Node1/ />
<USERLIST>C:/joeapp/ODB_Nano/Nodes/oodb_Node1/userlist />
```

Note: ODBService automatically searches for your computer's assigned IP addresses. These are typically listed as IPv4 and IPv6. If you assign the following to the key:

- <WEB_HOST/IP>**IP6**@9999 /> **IPv6** will be searched for and used, and the port is 9999.
- <WEB_HOST/IP>**IP4**@9999 /> **IPv4** will be searched for and used, and the port is 9999.
- <WEB_HOST/IP>localhost@9999 /> localhost (or 127.0.0.1) is the WebHostName, and the port is 9999.